Apply filters to SQL queries

Project description

As a security analyst, I investigated several potential security incidents by querying a database to identify unauthorized or suspicious activity. Using SQL filters, I extracted specific data points related to failed login attempts, geographic origins of traffic, and employee department locations. This process allowed me to refine large datasets into actionable insights, ensuring that security updates and investigations were targeted and efficient.

Table formats

This document describes how the tables used for this portfolio activity are organized. The `organization` database contains the following two tables:

- `log_in_attempts`
- `employees`

log_in_attempts

The `log_in_attempts` table has the following columns:

- `event_id`: The identification number assigned to each login event
- `username`: The username of the employee
- `login_date`: The date the login attempt was recorded
- `login_time`: The time the login attempt was recorded
- `country`: The country where the login attempt occurred
- `ip_address`: The IP address of that employee's machine
- `success`: The success of the login attempt; `FALSE` indicates a failed attempt

In the MariaDB shell, these columns are returned as:

```
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
```

employees

The `employees` table has the following columns:

- `employee_id`: The identification number assigned to each employee
- `device_id`: The identification number assigned to each device used by the employee
- `username`: The username of the employee
- `department`: The department the employee is in
- `office`: The office the employee is located in

In the MariaDB shell, these columns are returned as:

```
+-------------+--------------+----------+------------------------+-------------+
| employee_id | device_id    | username | department             | office      |
+-------------+--------------+----------+------------------------+-------------+
```

Retrieve after-hours failed login attempts

Query:

```
SELECT *
FROM log_in_attempts
WHERE login_time > '18:00' AND success = 0;
```

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE login_time > '18:00' AND success = FALSE;
+----------+----------+------------+------------+---------+-----------------+-
------+
| event_id | username | login_date | login_time | country | ip_address      |
ccess |
+----------+----------+------------+------------+---------+-----------------+-
------+
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |
    0 |
|       18 | pwashing | 2022-05-11 | 19:28:50   | US      | 192.168.66.142  |
    0 |
|       20 | tshah    | 2022-05-12 | 18:56:36   | MEXICO  | 192.168.109.50  |
    0 |
|       28 | aestrada | 2022-05-09 | 19:28:12   | MEXICO  | 192.168.27.57   |
    0 |
```

How it works: This query uses the WHERE clause with the AND operator to find rows where two conditions are met: the login_time is after 18:00 and the success value is 0 (representing a failure).

Analysis: The query successfully identified 19 failed login attempts that occurred after business hours. This specific set of data allows the security team to investigate whether these attempts represent a brute-force attack or unauthorized access patterns.

Retrieve login attempts on specific dates

Query:

```
SELECT * FROM log_in_attempts
WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
+----------+----------+------------+------------+---------+-----------------+-
------+
| event_id | username | login_date | login_time | country | ip_address      |
ccess |
+----------+----------+------------+------------+---------+-----------------+-
------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |
    1 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |
    1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |
    0 |
|        8 | bisles   | 2022-05-08 | 01:30:17   | US      | 192.168.119.173 |
    0 |
|       12 | dkot     | 2022-05-08 | 09:11:34   | USA     | 192.168.100.158 |
    1 |
```

How it works: I used the OR operator to filter for two specific dates. This ensures that any login attempt occurring on either the day of the suspicious event or the day prior is included in the results.

Analysis: This query returned 75 login attempts across those two days. Reviewing this volume of activity helps determine if the suspicious activity on the 9th was an isolated incident or part of a multi-day trend starting on the 8th.

Retrieve login attempts outside of Mexico

Query:

```
SELECT * FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE NOT country LIKE 'MEX%';
+----------+----------+------------+------------+---------+-----------------+--
------+
| event_id | username | login_date | login_time | country | ip_address      | s
uccess |
+----------+----------+------------+------------+---------+-----------------+--
------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |
    1 |
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |
    0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |
    1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |
    0 |
|        5 | jrafael  | 2022-05-11 | 03:05:59   | CANADA  | 192.168.86.232  |
    0 |
```

How it works: This query uses NOT combined with LIKE and the % wildcard. The pattern 'MEX%' catches both "MEX" and "MEXICO," and the NOT operator excludes them to show all other countries.

Analysis: The filter identified 144 login attempts made outside of Mexico. Excluding known-safe or irrelevant regions helps the team focus on potential external threats originating from other locations.

Retrieve employees in Marketing

Query:

```
SELECT * FROM employees
WHERE department = 'Marketing' AND office LIKE 'East%';
```

```
MariaDB [organization]> SELECT * FROM employees
    ->
    -> WHERE department = 'Marketing' AND office LIKE 'East%';
+-------------+-------------+----------+------------+----------+
| employee_id | device_id   | username | department | office   |
+-------------+-------------+----------+------------+----------+
|        1000 | a320b137c219 | elarson  | Marketing  | East-170 |
|        1052 | a192b174c940 | jdarosa  | Marketing  | East-195 |
|        1075 | x573y883z772 | fbautist | Marketing  | East-267 |
|        1088 | k8651965m233 | rgosh    | Marketing  | East-157 |
|        1103 | NULL         | randerss | Marketing  | East-460 |
|        1156 | a184b775c707 | dellery  | Marketing  | East-417 |
|        1163 | h679i515j339 | cwilliam | Marketing  | East-216 |
+-------------+-------------+----------+------------+----------+
7 rows in set (0.019 sec)
```

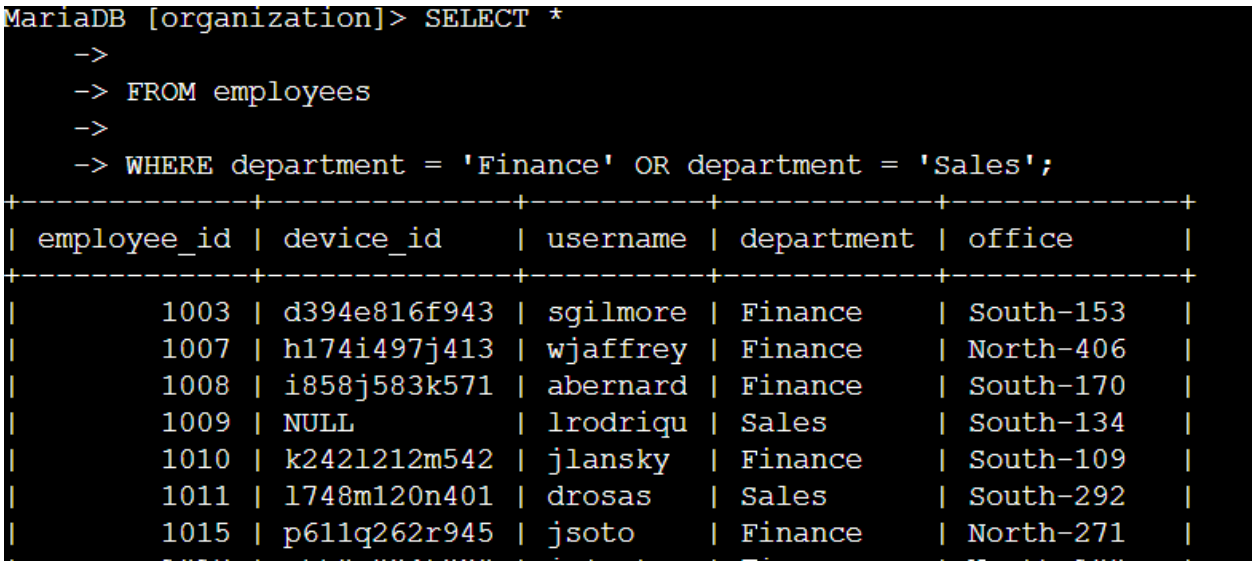
How it works: I applied the AND operator to ensure both the department and office location matched. The LIKE 'East%' filter captures all offices located in the East building (e.g., East-170, East-320).

Analysis: This query provided a targeted list of Marketing employees in the East building, ensuring that the security update team knows exactly which machines to visit without disrupting other departments or buildings.

Retrieve employees in Finance or Sales

Query:

```
SELECT *
FROM employees
WHERE department = 'Finance' OR department = 'Sales';
```



```
MariaDB [organization]> SELECT *
    ->
    -> FROM employees
    ->
    -> WHERE department = 'Finance' OR department = 'Sales';
+-------------+--------------+----------+------------+-------------+
| employee_id | device_id    | username | department | office      |
+-------------+--------------+----------+------------+-------------+
|        1003 | d394e816f943 | sgilmore | Finance    | South-153   |
|        1007 | h174i497j413 | wjaffrey | Finance    | North-406   |
|        1008 | i858j583k571 | abernard | Finance    | South-170   |
|        1009 | NULL         | lrodriqu | Sales      | South-134   |
|        1010 | k242l212m542 | jlansky  | Finance    | South-109   |
|        1011 | l748m120n401 | drosas   | Sales      | South-292   |
|        1015 | p611q262r945 | jsoto    | Finance    | North-271   |
```

How it works: Using the OR operator allows the query to return rows for employees in either the Finance department or the Sales department.

Analysis: This identifies all staff across both departments who require the secondary security update, allowing for efficient scheduling of maintenance across these two specific business units.

Retrieve all employees not in IT

Query:

```
SELECT *
FROM employees
WHERE NOT department = 'Information Technology';
```

```
MariaDB [organization]> SELECT *
    ->
    -> FROM employees
    ->
    -> WHERE NOT department = 'Information Technology';
+-------------+-------------+----------+-----------------+-------------+
| employee_id | device_id   | username | department      | office      |
+-------------+-------------+----------+-----------------+-------------+
|        1000 | a320b137c219 | elarson  | Marketing       | East-170    |
|        1001 | b239c825d303 | bmoreno  | Marketing       | Central-276 |
|        1002 | c116d593e558 | tshah    | Human Resources | North-434   |
|        1003 | d394e816f943 | sgilmore | Finance         | South-153   |
|        1004 | e218f877g788 | eraab    | Human Resources | South-127   |
|        1005 | f551g340h864 | gesparza | Human Resources | South-366   |
```

How it works: The NOT operator is used to filter out the Information Technology department, leaving all other departments in the results.

Analysis: The query listed 161 employees who are not in the IT department. Since the IT department had already been updated, this result provides a complete list of all remaining staff who still require the mandatory security update.

Summary

I utilized SQL filtering techniques, including AND, OR, NOT, and LIKE to extract critical security and employee data. These tasks allowed me to identify 19 high-risk after-hours login failures, analyze 75 attempts over a specific 48-hour window, and filter 144 international attempts. Finally, I generated a list of 161 employees requiring system updates, ensuring the organization's security protocols were applied comprehensively across all non-IT departments.